



Final exam with solutions

Introduction to python (Universiteit van Amsterdam)



Scan to open on Studocu

Question 1 – 6–OOP–1 – 308088.1.2

Take a look at the following class definition.

```
class Flight:
    def __init__(self, airline, origin, destination, capacity=300):
        self.airline = airline
        self.origin = origin
        self.destination = destination
        self.capacity = capacity

    def set_date(self, date):
        self.date = date
```

Given the class definition above, trying to execute the following two code lines will result in a Python error.

```
my_flight = Flight("KLM", "Amsterdam", "Paris")      # Code line 1
my_flight.set_date(my_flight, "29-02-2022")          # Code line 2
```

Why?

- A** The first argument (`my_flight`) should not have been passed to the `set_date` method in Code line 2.
- B** The capacity of the flight is not specified as an argument in Code line 1.
- C** The argument `self` is missing in Code line 1.
- D** The date string passed as an argument in Code line 2 represents an invalid date.

Question 2 – 6-OOP-2 – 308184.1.4

You have the following definition for a **Book** class.

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.review_scores = []

    def add_review(self, score):
        self.review_scores.append(score)

    def num_reviews(self):
        return len(self.review_scores)

    def average_score(self):
        if self.num_reviews() > 0:
            return sum(self.review_scores) / len(self.review_scores)
```

Suppose that you also want to write a method called **compare** that compares the book to another book and gives a recommendation for which one to read. A book is better than another book if its average review score is higher than the second book's and it has at least as many reviews as the second book.

The **compare** method should return the title of the better book. If neither book is better than the other, then it should return `None`.

For simplicity, you can assume that both books have at least one review already.

Which method definition achieves what you want?

- A**
- ```
def compare(self, other):
 avg_better = self.average_score() > other.average_score()
 no_fewer_reviews = self.num_reviews() >= other.num_reviews()
 if avg_better and no_fewer_reviews:
 return self.title
 elif not avg_better:
 return other.title
 else:
 return None
```
- B**
- ```
def compare(self, other):
    for first, second in [(self, other), (other, self)]:
        avg_better = first.average_score() > second.average_score()
        no_fewer_reviews = first.num_reviews() >= second.num_reviews()
        if avg_better and no_fewer_reviews:
            return first.title
```
- C** Both methods above compare the book objects as intended.
- D** Neither of the two methods above compare the book objects as intended.

Question 3 – 6-strings-1 – 308059.1.1

You would like to write a function called `get_tld` that extracts the top-level domain from the URL of a website. The function takes the URL as the input string in the following format:

`"https://www.[SECOND-LEVEL DOMAIN].[TOP-LEVEL DOMAIN]/[FOLDER 1]/[...]/[FOLDER N]/[PAGE NAME].html"`
and returns the top-level domain as a string.

For example, the function call:

```
get_tld("https://www.uva.nl/en/education/bachelor-s/bachelors.html")  
should return the string:  
"nl"
```

Which of the following code segments would **not** achieve what you want?

- A**

```
def get_tld(url):  
    url_1 = url.split(".")[-1]  
    return url_1.split("/") [0]
```
- B**

```
def get_tld(url):  
    url_1 = url.split("/") [1]  
    url_2 = url_1.split("/") [0]  
    return url_2.split(".") [-1]
```
- C**

```
def get_tld(url):  
    return url.split("/") [1].split("/") [0].split(".") [-1]
```
- D**

```
def get_tld(url):  
    url_1 = url[url.find("/") + 2:]  
    url_2 = url_1.split("/") [0]  
    return url_2.split(".") [-1]
```

Question 4 – 6-strings-2 – 308060.1.1

You're trying to write a function called `is_anagram`, which takes two strings as inputs and checks whether they are anagrams of each other. That is: whether one of them can be written by rearranging the letters of the other. Upper case and lower case versions of a letter count as the same letter, and whitespace characters don't count at all.

For example, the following function calls should all return the boolean value `True`:

```
is_anagram("Old West Action", "Clint Eastwood")  
is_anagram("eleven plus two", "twelve plus one")  
is_anagram("I am a weakish speller", "William Shakespeare")
```

while the following function call should return the boolean value `False`:

```
is_anagram("one plus two", "three")
```

Which of the following code segments achieves what you want?

- A**

```
def is_anagram(word_1, word_2):  
    return sorted(list(word_1.replace(" ", "").lower())) == sorted(list(word_2.replace(" ",  
    "").lower()))
```
- B**

```
def is_anagram(word_1, word_2):  
    return word_1.replace(" ", "").lower().sort() == word_2.replace(" ", "").lower().sort()
```
- C**

```
def is_anagram(word_1, word_2):  
    return list(word_1.replace(" ", "").lower()).sort() == list(word_2.replace(" ",  
    "").lower()).sort()
```
- D**

```
def is_anagram(word_1, word_2):  
    return sorted(list(word_1)).replace(" ", "").lower() == sorted(list(word_2)).replace(" ",  
    "").lower()
```

Question 5 – 7–pandas–1 – 308194.1.0

You have a pandas Series called **s**, which contains floats. You'd like to create a new Series, in which each element is equal to the square of the difference between the corresponding element in **s** and the mean of the elements in **s**.

For example, if **s** is the Series:

```
0    2.0
1    2.0
2    3.0
3    5.0
dtype: float64
```

then you want the new Series to be:

```
0    1.0
1    1.0
2    0.0
3    4.0
dtype: float64
```

Which of the following code segments does **not** achieve what you want?

- A** `s.map((s - s.mean())**2)`
- B** `(s - s.mean())**2`
- C** `s.map(lambda x: (x - s.mean())**2)`
- D** `(s - s.mean()).map(lambda x: x**2)`

Question 6 – 7–pandas–2 – 308216.1.1

You have a pandas DataFrame called **df**. It has a column called "A" and a column called "B". Both contain numbers.

You'd like to create a column called "C", which contains the difference of the numbers in columns "A" and "B". Which of the following code lines will achieve what you want?

- A** `df["C"] = df["A"] - df["B"]`
- B** `df["C"] = df["A"].map(lambda x: x - df["B"])`
- C** `df["C"] = df.columns["A"] - df.columns["B"]`
- D** `df["C"] = df.apply(df["A"] - df["B"])`

Question 7 – 7-pandas-3 – 308219.2.1

You have a pandas DataFrame called **df** that looks like this, when printed:

	A	B	C	D
1	8.7	4.6	9.5	7.0
2	7.3	1.5	6.3	7.1
3	4.3	9.0	2.9	3.3
4	4.8	2.5	5.1	3.0
5	1.6	1.4	6.7	3.5
6	2.4	3.7	7.6	1.3

Suppose that you'd like to create a new Series that only contains those elements of column "D" of **df** that are smaller than 5.0. So the Series should look like this, when printed:

3	3.3
4	3.0
5	3.5
6	1.3

Name: D, dtype: float64

Which of the following code lines will **not** achieve what you want?

- A** `df.loc[2:6, df.columns[-1]]`
- B** `df.loc[df["D"] < 5, "D"]`
- C** `df.loc[3:6, "D"]`
- D** `df.iloc[2:6, 3]`

Question 8 – 7-pandas-4 – 308231.1.0

You have two DataFrames: **df1** and **df2**. They look like this when printed:

```
print(df1)
```

	Age	Height	Female	City	Language
0	23	167	True	Berlin	German
1	36	175	True	Frankfurt	German
2	17	182	False	Antwerpen	Dutch
3	25	177	True	Eindhoven	Dutch
4	36	178	False	Lyon	French
5	52	185	False	Basel	German
6	46	168	True	Innsbruck	German

```
print(df2)
```

	Age	Height	Female	City	Language
5	52	185	False	Basel	German
4	36	178	False	Lyon	French
1	36	175	True	Frankfurt	German
6	46	168	True	Innsbruck	German
0	23	167	True	Berlin	German

Which of the following code segments could have created **df2** from **df1**?

- A** `df1.loc[df1["Language"].isin(["French", "German"])].sort_values("Height", ascending=False)`
- B** `df1.loc[df1["Language"] != "Dutch"].sort_values("Age", ascending=False)`
- C** `df1.loc[~df1["Language"].isin(["Dutch"]), ["Age", "Height", "City", "Language"]].sort_index(ascending=False)`
- D** `df1.loc[df1["Language"] != "Dutch"].sort_index(axis=1)`

Question 9 – 8-comprehensions-1 – 308232.1.3

You have a list of mountain peaks and their heights in meters, all formatted as "[PEAK NAME]: [HEIGHT]m". For example, a few elements of the list are:

```
"Denali: 6,190m", "Aconcagua: 6,961m", "Kilimanjaro: 5,895m", ...
```

As you can see, the comma (",") is used as a thousand separator in the height. The list is called **mountains**.

You'd like to create a dictionary called **peaks** from the **mountains** list. The peak names would be the keys, and their heights the corresponding integer values. Continuing the example above, a few key-value pairs of the **peaks** dictionary should be:

```
"Denali": 6190, "Aconcagua": 6961, "Kilimanjaro": 5895, ...
```

Which of the following two code segments achieves what you want?

- A** `peaks = {peak.split(":")[0]: int(peak.split(": ")[1][:-1].replace(",", "")) for peak in mountains}`
- B** `names = [peak.split(":")[0] for peak in mountains]
heights = [int(peak.split(": ")[1].replace(",", "").replace("m", "")) for peak in mountains]
peaks = dict(zip(names, heights))`
- C** Both code segments above achieve the intended outcome.
- D** Neither of the two code segments above achieves the intended outcome.

Question 10 – 8-comprehensions-2 – 308234.1.2

You have a dictionary called **grades**, in which the keys are unique student ID's (as strings) and the corresponding values are each student's Python course grades (as floats).

You'd like to create another dictionary called **grade_curve**, in which the keys are the 18 possible Dutch course grades (1.0, 1.5, 2.0, ..., 5.0, 6.0, ..., 10) as floats, and the corresponding (integer) values are the number of students in **grades** who have that particular grade.

Take a look at the following code segment to create the **grade_curve** dictionary.

```
dutch_grades = [1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0,
                6.0, 6.5, 7.0, 7.5, 8.0, 8.5, 9.0, 9.5, 10.0]
grade_curve = {grade: sum([v == grade for v in grades.values()]) for grade in dutch_grades}
```

Do you think this code segment will always work as intended? If not, then why not?

- ☒ **A** Yes, the code will always work as intended.
- ☐ **B** The code will not work as intended if there are certain grades that no student has received. That will lead to an error in constructing the dictionary comprehension.
- ☐ **C** The code will not work as intended, because the keys of the **grades** dictionary are not used in the list comprehension inside the **sum** function.
- ☐ **D** The code will not work as intended, because including a list comprehension inside a dictionary comprehension will lead to a Python syntax error.

Question 11 – 8-datetime-1 – 308235.1.0

What does the following code segment print to the screen?

```
from datetime import datetime, timedelta

date = datetime.strptime("04.05.2020", "%m.%d.%Y")
print((date + timedelta(days=-10)).strftime("%d-%m-%Y"))
```

- ☒ **A** 26-03-2020
- ☐ **B** 04-24-2020
- ☐ **C** 15-04-2020
- ☐ **D** 05-14-2020

Question 12 – 8-datetime-2 – 308243.1.4

Suppose that you have the following definition for a **Meeting** class, which records basic information about meetings in a calendar application.

```
from datetime import datetime, timedelta

class Meeting:
    def __init__(self, topic, date, start_time="09:00", length=30):
        self.topic = topic
        self.date = datetime.strptime(date, "%d-%m-%Y")
        self.start = datetime(self.date.year, self.date.month, self.date.day,
                               int(start_time.split(":")[0]), int(start_time.split(":")[1]))
        self.length = timedelta(minutes=length)
        self.end = self.start + self.length
```

You would like to add a method called **lunch_meeting** to the class, which returns a boolean value indicating whether there is any overlap between the official lunchtime (12:30 - 13:00) and the time of the meeting.

For example, if you construct a **Meeting** object as:

```
my_meeting = Meeting("Ask for salary raise", "25-05-2022", start_time="12:00", length=45)
```

then the method call:

```
my_meeting.lunch_meeting()
```

should return the boolean value **True**.

Which of the following code segments will achieve what you want?

- A**
- ```
def lunch_meeting(self):
 lunch_start = datetime(self.date.year, self.date.month, self.date.day, 12, 30)
 lunch_end = lunch_start + timedelta(minutes=30)
 return self.start < lunch_end and self.end > lunch_start
```
- B**
- ```
def lunch_meeting(self):
    return self.start < "13:00" and self.end > "12:30"
```
- C**
- ```
def lunch_meeting(self):
 return start_time < "13:00" and start_time + length > "12:30"
```
- D**
- ```
def lunch_meeting(self):
    lunch_start = datetime(self.date.year, self.date.month, self.date.day, 12, 30)
    lunch_end = datetime(self.date.year, self.date.month, self.date.day, 13, 0)
    return self.start >= lunch_end or self.end <= lunch_start
```

Question 13 – Basic logic 2 – 306992.1.2

Suppose you have a variable **x** which contains a list of tuples, as follows:

```
x = [(1, 2), (3, 4), (5, 6), (7, 8)]
```

Choose the correct output for the following line of code.

```
(x[0][1] * x[-3][0]) == (x[-1][1] - x[-4][1])
```

- A** True
- B** False
- C** NameError
- D** SyntaxError

Question 14 – Basic loop 2 – 306995.4.1

Which of the following code segments will print every even number between 0 and 20 (including 0 and 20) to the screen?

- A** `for i in range(0, 20):`
 `if i % 2 == 0:`
 `print(i)`
- B** `i = 0`
 `while i <= 20:`
 `if i % 2 == 0:`
 `print(i)`
 `i += 1`
- C** `for i in list(range(0, 20)):`
 `print(i if i % 2 == 0 else None)`
- D** `i = 0`
 `while i < 21:`
 `print(i)`
 `i = i + 1`

Question 15 – Basic scope 2 – 307002.2.1

Choose the correct statement about the program below:

```
a = 1
def outer(b):
    b = 2
    def inner(c):
        d = 3
        return a + c + d
    return inner(4) * b
```

- A** **a** is a global variable. **b**, **c**, **d** are local variables.
- B** **a**, **b** are global variables. **c**, **d** are local variables.
- C** **a**, **b**, **c**, **d** are global variables.
- D** **a**, **c**, **d** are global variables. **b** is a local variable.

Question 16 – final-dictionary-1 – 307012.2.12

You have to write a function called **main**, which takes a list of unique integers as an argument and returns a dictionary.

The output dictionary should consist of all key-value pairs for which the key is an element of the input list and the value is the next element of the input list. When there is no "next element" any more, leave the key-value pair out of the dictionary.

For example, if your function is called as:

```
main([1, 3, 2, 4])
```

then it should return the dictionary:

```
{1: 3, 3: 2, 2: 4}
```

Which of the following programs work(s) according to the requirements?

A

```
def main(l1):
    result = {}
    index = 0
    while True:
        index = index + 1
        if index >= len(l1):
            break
        else:
            result[l1[index]] = l1[index + 1]
    return result
```

B

```
def main(l1):
    result = {}
    for index, value in zip(l1[:-1], l1[1:]):
        result[index] = value
    return result
```

C Both programs work as required

D Neither of the two programs works as required

Question 17 – final-dictionary-2 – 308086.1.4

What will be printed by the following program?

```
d1 = {1: 5, 4: 6}
d2 = {4: 6, 1: 5}
print(d1 == d2)
```

A True

B False

C An error message

D None

Question 18 – final-dictionary-3 – 308087.2.7

```
d1 = {1: 2, 2: 1}
```

The following code segments all print exactly the same output, except for one. Which one?

- A** `result = 1`
 `for x in d1.values():`
 `result *= x`
 `print(result)`
- B** `result = -1`
 `for x in d1.keys():`
 `result += d1[x]`
 `print(result)`
- C** `result = 4`
 `for x in d1.items():`
 `result = result // x[1]`
 `print(result)`
- D** `result = -4`
 `for x in d1:`
 `result /= d1[x]`
 `print(result)`

Question 19 – final-functions-1 – 308082.1.8

Suppose that you have a string called **s1**, which contains several letters. The letter "x" and the letter "y" occur exactly once in the string, and "x" always precedes "y".

You'd like to print out a string that looks just like **s1**, except that the letters "x" and "y" should be swapped.

For example, if:

```
s1 = "axbyc"
```

then you'd like to print:

```
aybxc
```

Reminder: The replace string method has a third argument, which allows you to specify the maximum number of replacements you want to make, counting from left to right.



- A** `s1 = s1.replace('x', 'y', 1).replace('y', 'x', 1)`
 `print(s1)`
- B** `s1 = s1.replace('y', 'x', 1).replace('x', 'y', 1)`
 `print(s1)`
- C** `s1.replace('x', 'y', 1).replace('y', 'x', 1)`
 `print(s1)`
- D** `s1.replace('y', 'x', 1).replace('x', 'y', 1)`
 `print(s1)`

Question 20 – final-functions-2 – 308085.3.2

Take a look at the following function definition.

```
def glue(**kwargs):  
    result = ""  
    for el in kwargs.keys():  
        result = el + result  
    return result
```

What will be returned by the following function call:

```
glue(a="c", b="d")
```

A "ab"

B "cd"

C "ba"

D "dc"

Question 21 – final-functions-3 – 308245.3.2

Take a look at the following function definition.

```
def tester(i1 = 0):  
    if i1 >= 0:  
        return 1  
    if i1 <= 0:  
        return -1
```

What will be printed to the screen by the following code?

```
print(tester())
```

A 1

B -1

C 0

D None

Question 22 – List lambda 2 – 306736.2.9

Assume that you have a variable called **x**, which contains a list of integers.

Which of the following code segments will create another list that has every element of **x** multiplied by 2?

For example, if

```
x = [1, 2, 3, 4, 5]
```

then which of the following code segments will result in the list:

```
[2, 4, 6, 8, 10]
```

- A** `list(map(lambda i: i * 2, x))`
- B** `[(lambda i: i * 2)(item) for item in x]`
- C** `y = lambda i: i * 2`
 `[y(j) for j in x]`
- D** All three code segments would work as intended

Question 23 – List logic 2 – 306795.1.7

Which of the following programs creates a new list **y**, in which the elements are those elements of list **x** that are equal to their indices? You can assume **x** consists of unique integers.

For example, if:

```
x = [0, 2, 4, 3, 5, 9, 6, 7, 8, 1, 10]
```

then the program should create a new list **y** such that:

```
y = [0, 3, 6, 7, 8, 10]
```

- A** `y = []`
 `for i, j in enumerate(x):`
 `if i == j:`
 `y.append(i)`
- B** `y = []`
 `for i, j in enumerate(x):`
 `if i == j:`
 `y.append(j)`
- C** `y = [i for i in x if i == x.index(i)]`
- D** All three programs would work as intended

Question 24 – List loop 2 – 306787.2.5

Assume that you have a list called **x**, which is a list of multiple sub-lists containing integers.

Which of the following programs will create a new list **y** that is equal to the sub-list of **x** whose sum of elements is the highest?

You can assume that the sum of elements in each sub-list is a different number, so that there is only one that is the highest.

For example, if:

```
x = [[8, 20, 300], [7, 8, 9], [10, 11, 12], [40, 5, 6]]
```

then **y** should be:

```
y = [8, 20, 300]
```

A `y = max(x)`

B `z = []
for i in x:
 z.append(sum(i))
y = x[z.index(max(z))]`

C `x.sort()
y = x[-1]`

D `y = [i for i in x if sum(i) == max(x)]`